

Course	COMP 8005
Program	Bachelor of Science in Applied Computer Science
Term	January 2026

- This is an individual [programming](#) assignment.

Objective

Design and implement a distributed password-cracking system for UNIX systems using a controller multi-worker architecture.

You will:

- Measure performance and overheads of distribution.
- Study scaling across multiple workers (inter-host) and threads within each worker (intra-worker).
- Use measured results for 1–3 workers to predict performance at 5 workers, then validate by running 5 workers.

System Requirements

Components

Your system must consist of:

- A controller program running on Host A.
- N worker programs running on other hosts.

Worker Threads

Each worker is multithreaded and supports a configurable thread count via the command line.

Communication Model

- No shared memory between hosts.
- Use a network protocol of your choice.
- Minimize unnecessary communication.
- Workers request work using a pull model (workers ask, controllers reply).

Learning Outcomes

By completing this assignment, you will be able to:

- Design and implement a controller / multi-worker distributed system.
- Implement worker registration, remote job execution, and work assignment.
- Correctly parse UNIX shadow entries and verify candidate passwords.
- Implement a multi-threaded search with correct partitioning and no duplicate coverage within assigned work.
- Implement controller-driven heartbeat and progress reporting.
- Implement checkpointing for recoverability and measure its overhead.
- Measure and analyze end-to-end execution time and overhead across distributed systems.
- Measure and analyze scaling behaviour for 1–3 workers and validate a 5-worker prediction.
- Work with multiple UNIX password hashing algorithms.
- Present experimental results using tables, graphs, and written analysis.

Supported Password Hashing Algorithms

Your system must support all of the following UNIX password hashing algorithms:

- yescrypt
- bcrypt
- SHA256
- SHA512
- MD5

Password Search Parameters

All experiments must use:

- A 3-character password
- The 79-character legal character set

The entire search space must be covered such that:

- The system completes the search (or finds the password), and
- The system completes any unfinished work after a worker's failure.

Fault Model

You must support worker crash recovery.

If a worker crashes:

- The worker may lose the in-memory state.

- The controller must detect a loss of liveness (via heartbeat) and ensure that the unfinished portion of the work is completed by reassigning it.
- A worker that crashes and restarts must be able to resume from its own local checkpointing, subject to whatever work the controller assigns.

Important constraints:

- You are not required to implement any specific crash recovery technique.
- You are not required to use any specific data structure, database, lease system, or consensus method.
- The only required coupling is that the controller accepts a checkpoint interval command-line argument and passes that value to the workers.

Architecture

Controller

There is exactly one controller.

The controller is responsible for:

1. Input parsing
 - Accept a shadow file and username from the command line.
 - Parse the selected shadow entry to extract:
 - hashing algorithm identifier
 - salt
 - password hash
2. Worker lifecycle
 - Accept registrations from multiple workers.
 - Send full cracking job parameters to workers:
 - algorithm, salt, hash
 - search parameters (including how your system represents the search space)
 - chunk/work-unit size
 - heartbeat interval
 - checkpoint interval (see Checkpointing)
3. Work allocation
 - Manage a global allocator that issues disjoint work units (chunks or equivalent).
 - Ensure the system can complete the entire search space, even if a worker crashes.
4. Heartbeats

- Send heartbeat requests every N seconds (configurable).
 - Receive heartbeat responses with progress information.
5. Checkpoint coordination (required)
 - Accept a checkpoint interval from the command line.
 - Send that checkpoint interval to every worker.
 - Receive checkpoint reports from workers.
 6. Crash recovery
 - If a worker becomes unresponsive (as indicated by a lack of heartbeat), ensure unfinished work can still be completed.
 7. Global stop
 - Once any worker finds the password:
 - No new work is issued
 - The controller sends STOP to the workers so they can exit
 8. Reporting
 - Final result
 - Timing breakdowns
 - Experimental measurements

Workers

There are N workers.

Each worker is responsible for:

1. Registration
 - Start and register with the controller.
 - Receive job parameters.
2. Work execution
 - Repeatedly request work (pull model):
 - ask for a work unit
 - crack the assigned unit using T threads
 - report FOUND immediately if discovered
 - otherwise request the next unit
3. Heartbeats
 - Respond to controller heartbeats with progress since the last heartbeat.
4. Checkpointing
 - Send checkpoints to the controller at the interval specified by the controller.

5. Exit

- Exit when STOP is received, or when the job is complete/exhausted.

Work Assignment Model

Distribution level (between hosts)

- The controller partitions the global search space into work units (chunks or equivalent).
- A work unit corresponds to some number of candidate passwords (configurable).
- Workers request new work when done.

Worker level (within a worker process)

Within a worker, the assigned work must be partitioned across T threads such that:

- Each candidate in the worker's assigned work is tested exactly once by that worker.
- No duplicates within the assigned work.
- No omissions within the assigned work.

Acceptable partitioning approaches include:

- Static partitioning (each thread gets a disjoint subrange)
- Dynamic partitioning (preferred): a thread-safe work index / internal chunking

Correctness Requirements

Global correctness

Your system must guarantee:

- The search completes correctly in the no-failure case.
- If a worker crashes, the system still completes the search by ensuring unfinished work is eventually attempted.
- Once FOUND occurs, the system stops issuing new work, and workers exit.

Worker correctness

Within a worker:

- No candidate in an assigned work unit may be tested twice.
- No candidate in an assigned work unit may be omitted.

Heartbeat Protocol

The heartbeat provides liveness + progress visibility during long runs.

Behavior:

- The controller sends a heartbeat request every N seconds.
- Each worker replies with the number of candidates tested since the last heartbeat.

Minimum required heartbeat response payload:

- `delta_tested`: number of candidate attempts since the last heartbeat

Recommended (optional) fields:

- `total_tested`
- `threads_active`
- `current_rate` (attempts/sec since last heartbeat)
- identifier for current work unit/position

Important:

- Heartbeat must not distort correctness. It is for measurement and observability, not control.

Checkpointing

Controller-configured checkpoint interval

The controller must accept a checkpoint interval from the command line:

- checkpoint every K password attempts

The controller must send this checkpoint interval to workers as part of the job parameters.

Implementation constraint:

- The only required implementation detail is: the controller takes a checkpoint interval argument and sends that value to workers.

Worker-to-controller checkpoints

Workers must send checkpoint reports to the controller at the configured interval.

Minimum required checkpoint payload:

- Enough information for the controller to understand the worker's progress within its assigned work.

Stop Conditions

If any worker finds the password:

1. Worker reports FOUND to the controller immediately.
2. Controller sets a found flag.
3. No new work is assigned.
4. Controller sends STOP to workers.
5. Workers exit.

Command Line Interface

Controller

The controller must be runnable as:

```
controller -f <shadow file> -u <username> -p <port> -b  
<heartbeat_seconds> -c <chunk_size> -k <checkpoint_attempts>
```

Required arguments:

- -f path to shadow file (/etc/shadow or a copy)
- -u username whose password will be cracked
- -p port number to listen on
- -b heartbeat interval in seconds
- -c chunk size in candidates (passwords) per assigned work unit
- -k checkpoint interval in candidate attempts between worker-to-controller checkpoints

Worker

Each worker must be runnable as:

```
worker -c <controller_host> -p <port> -t <threads>
```

Required arguments:

- -c controller hostname or IP address
- -p controller port number
- -t number of worker threads to use for cracking

Performance Experiments

Required worker counts

You must run the full experiment suite at:

- 1 worker
- 2 workers
- 3 workers
- 10 workers (prediction validation)

Required thread count per worker

Use a fixed thread count per worker for worker-scaling experiments:

- $t = 1$ (pure distribution scaling), or
- $t = \text{number of CPU cores} - 1$

Pick one and use it consistently across scaling experiments.

Required measurements (per trial)

For each hashing algorithm and worker count, measure and report:

- controller-side parsing time
- job dispatch/registration overhead
- work assignment overhead (per unit and/or total)
- worker cracking time (compute/search)
- result return latency (worker $\rightarrow$ controller)
- total end-to-end runtime
- heartbeat observations (progress per heartbeat; rate stability/variance)
- checkpoint overhead observations:
 - checkpoint frequency (as configured)
 - cost/impact (qualitative plus quantitative where possible)

Repetition and summary statistics

Run each experiment multiple times and report:

- mean
- min/max or standard deviation

Prediction Requirement (Worker Scaling)

After measuring 1–3 workers:

- Predict what will happen at 10 workers (runtime and/or speedup).
- Explain your model.

Acceptable approaches include:

- Amdahl-style model using measured serial fraction (controller + networking + checkpoint overhead)
- Throughput fits with diminishing returns

- Overhead-aware model including assignment, coordination, checkpoint, and scheduling costs

Then:

- Run the experiment with 10 workers
- Graph the measured result and explain why it did or did not match your prediction
- If it diverges, identify causes (controller bottleneck, network contention, chunk size, checkpoint interval too small/large, synchronization overhead within workers, OS scheduling, etc.)

Constraints Summary

- Exactly one controller
- Multiple workers; workers request work in units
- No shared memory between hosts
- Workers use T threads (configurable)
- Heartbeat is required and must not break correctness
- The controller checkpoint interval is configured on the controller and sent to workers
- Workers checkpoint to the controller and also check locally for self-resume
- Correctness across all supported hashing algorithms
- Minimize unnecessary communication

Resources

- UNIX crypt(3) documentation
- Password hashing libraries for your chosen language
- Socket/network programming documentation
- Profiling tools (time, perf, htop)

Submission

Ensure your submission meets all guidelines, including formatting, file type, and submission.

Follow the AI usage guidelines.

Be aware of the late submission policy to avoid losing marks.

Evaluation

Topic	Value
Design	10
Testing	25
Implementation	55
Report	10
Total	100

Testing Information

You should test with known passwords and algorithms, but be aware that during the final demo, you will not know:

- Password length
- Password hashing algorithm
- Password

Additionally, your implementation must demonstrate:

- correct operation with checkpointing enabled,
- continued progress toward completion when a worker becomes unresponsive and is effectively “lost.”
- worker's ability to restart and resume based on its local checkpointing (as applicable to the current job)